

Supply Chain AI Hybrid Orchestrator

End-to-End Enterprise Architecture & Operational Runbook

Target Environment: .NET eShop, Aspire, RabbitMQ, Rust Data Platform & Groq AI

Chapter 1: Architecture & The Hybrid Pattern

Autonomous LLMs present severe non-deterministic risks if given raw API write access. The Hybrid Architecture enforces a deterministic **PolicyGate** and feature layer around a sandboxed LLM reasoning agent (Groq / GPT-OSS-20B). The LLM is restricted to proposing quantities; execution is guarded by business invariant rules and human authorization webhooks.

Chapter 2: Host Prerequisites Checklist

Verify dependencies before running:

```
$ docker --version           # Container runtime
$ dotnet --version           # .NET 8 / 9 / 10 SDK
$ aspire version             # .NET Aspire CLI
$ cargo --version            # Rust Toolchain
$ python3 --version          # Python Virtual Environment
```

Chapter 3: Starting Aspire & Wave Load Generator

Execute from the repository root to start eShop and synthetic demand spikes:

```
cd ~/eShop
env 'Parameters__clickhouse-password=local-dev-only' \
DataPlatform__Enabled=true \
DataPlatform__AgentMode=AUTO_DEMO \
OrderGenerator__Enabled=true \
OrderGenerator__RatePerSecond=10 \
OrderGenerator__Continuous=true \
OrderGenerator__VariableRate=true \
OrderGenerator__WaveAmplitudeFraction=0.5 \
OrderGenerator__WavePeriodSeconds=120 \
OrderGenerator__MinimumRatePerSecond=5 \
OrderGenerator__Maximum_RATE_PER_SECOND=15 \
OrderGenerator__Profile=REGIONAL_SPIKE \
OrderGenerator__TargetSkuId=42 \
ESHOP_USE_HTTP_ENDPOINTS=1 \
aspire start --apphost src/eShop.AppHost/eShop.AppHost.csproj
```

Parameter	Configured Value	Operational Rationale
RatePerSecond	10	Base velocity: 600 orders/minute.
Continuous	true	Infinite loop testing.
VariableRate	true	Sinusoidal wave to simulate sudden rush.
Wave Bounds	5 to 15 /sec	Traffic oscillates between 300 and 900 orders/min.
Profile & SKU	REGIONAL_SPIKE / 42	Concentrates load onto NCR warehouse festive item.

Chapter 4 & 5: RabbitMQ Discovery & Rust Orchestrator Execution

Aspire binds containers to dynamic ports. Discover credentials and run the Rust binary:

```
# 1. Discover Port & Credentials
$ aspire describe eventbus --format Json

# 2. Export & Start Orchestrator
$ cd ~/eShop/src/RustDataPlatform
$ export AMQP_URL="amqp://guest:<PASSWORD>@127.0.0.1:<PORT>/%2f"
$ export GROQ_API_KEY="gsk_..."
```

```
$ export GROQ_MODEL="openai/gpt-oss-20b"
$ export LLM_TIMEOUT_MILLISECONDS=15000
$ export RUST_LOG="hybrid_orchestrator=info,info"
$ cargo run -p hybrid-orchestrator
```

Chapter 6 & 7: Demand Injection & Boss's Desk Approval

Spike events are delivered directly into RabbitMQ. If stock drops below threshold, the LLM creates a proposal. If the deficit is high, status transitions to **REQUIRES_HUMAN_APPROVAL**.

```
# Query proposals
$ curl -sS http://127.0.0.1:5005/api/v1/proposals | jq '.[0]'
```



```
# Authorize replenishment via Boss's Desk webhook
$ curl -X POST http://127.0.0.1:5005/api/v1/proposals/<PROPOSAL_ID>/approve | jq
```

Chapter 8: Troubleshooting Matrix

Symptom	Root Cause	Action
IOError: invalid port	Literal <PORT> in string	Read exact port from aspire describe eventbus.
Groq HTTP 404	Model ID deprecated	Use active model: export GROQ_MODEL='openai/gpt-oss-20b'.
Proposal REJECTED	Zero deficit	Ensure balance payload has onHand < safetyStock.
Groq HTTP 429	API rate limit reached	Circuit breaker automatically trips to deterministic fallback.